

```

// — Service worker: network-first for the app shell —
// Deploys are picked up immediately. Cache is only an offline fallback.
// Bump CACHE_VERSION on each deploy to purge old caches.
const CACHE_VERSION = 'zaor-studio-v1-99';
const CACHE = CACHE_VERSION;

self.addEventListener('install', e => {
  self.skipWaiting(); // activate immediately
});

self.addEventListener('activate', e => {
  e.waitUntil(
    caches.keys().then(keys =>
      Promise.all(keys.filter(k => k !== CACHE).map(k => caches.delete(k)))
    ).then(() => self.clients.claim())
  );
});

self.addEventListener('fetch', e => {
  // Let cross-origin requests (fonts, APIs) pass straight through
  if (!e.request.url.startsWith(self.location.origin)) return;

  const url = e.request.url;

  // NEVER cache the service worker script or manifest — if sw.js itself gets
  // cache-first served, the browser checks for updates against a stale copy
  // of itself and skipWaiting() never fires on new deploys. Always go to network
  if (url.endsWith('/sw.js') || url.endsWith('/manifest.json')) {
    e.respondWith(fetch(e.request).catch(() => caches.match(e.request)));
    return;
  }

  const isShell =
    e.request.mode === 'navigate' ||
    e.request.destination === 'document' ||
    url.endsWith('.html') ||
    url.endsWith('/');

  if (isShell) {
    // NETWORK-FIRST: always try fresh, fall back to cache only when offline
    e.respondWith(
      fetch(e.request)
        .then(res => {
          caches.open(CACHE).then(c => c.put(e.request, res.clone()));
        })
    );
  }

```

```
        return res;
    })
    .catch(() => caches.match(e.request))
);
} else {
    // CACHE-FIRST for other same-origin assets (fast, rarely change)
    e.respondWith(
        caches.match(e.request).then(cached =>
            cached || fetch(e.request).then(res => {
                caches.open(CACHE).then(c => c.put(e.request, res.clone()));
                return res;
            })
        ).catch(() => fetch(e.request))
    );
}
});
```